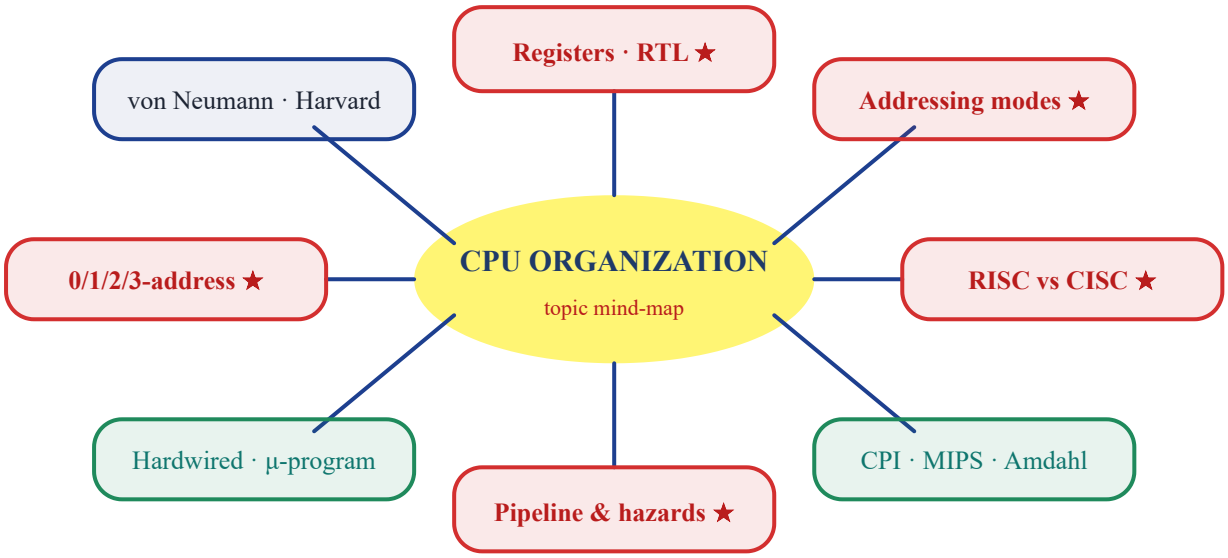
CPU Organization

Registers · RTL · addressing · RISC/CISC · pipelining · performance: exam-ready in 14 sheets



↑ the whole topic on one page: red = high-yield, green = method / numerical

Syllabus coverage

"Computer Architecture: … **Measuring performance**; … representing instruction, **addressing styles**; ALU operations, floating point operations, designing ALU; Processor design: datapaths & single cycle and multicycle implementations; **Control Unit design**: hardwired and microprogrammed; Hazards; Exceptions; **Pipeline** …, superscalar and dynamic pipelining; … channels." Cache and virtual memory are in the Memory Hierarchy lesson.

High-yield list

① PC · IR · MAR · MDR
② fetch cycle in RTL
③ EA for each mode
④ k + n − 1 cycles
⑤ RAW · forwarding
⑥ IC × CPI / f · Amdahl

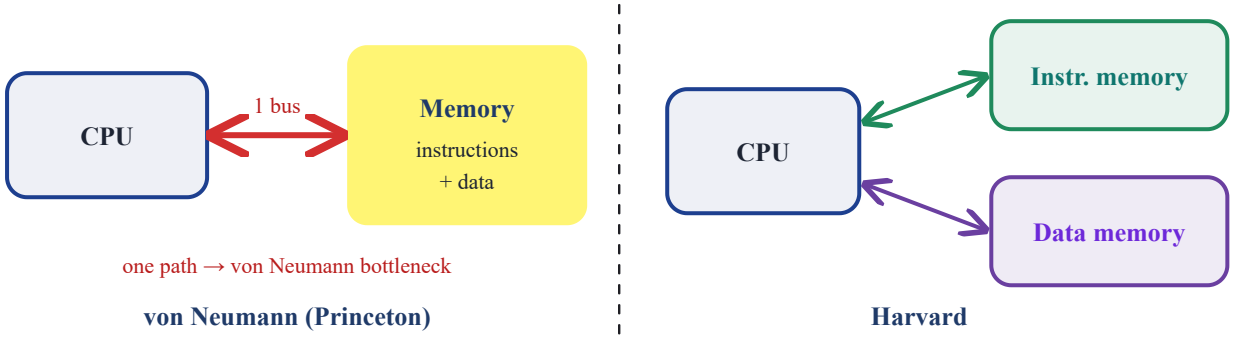
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ von Neumann, Harvard, CPU …… p. 2
✓ Buses, registers, flags …… p. 3
✓ Instruction cycle & RTL …… p. 4
✓ Instruction formats …… p. 5
✓ Addressing modes …… p. 6
✓ RISC/CISC, control unit …… p. 7
✓ Pipelining & speedup …… p. 8
✓ Hazards & their cures …… p. 9
✓ Performance & Amdahl …… p. 10
✓ ALU, fixed & floating point … p. 11
✓ Superscalar, Flynn, I/O …… p. 12
✓ Night-before sheet …… p. 13
✓ Practice MCQs + written …… p. 14

1 Stored Program: von Neumann vs Harvard

Stored-program concept: Instructions sit in the **same read-write memory as data** and are fetched and executed in sequence under the program counter. John von Neumann, **1945**, "First Draft of a Report on the EDVAC".



one shared memory + bus (left) vs separate instruction and data paths (right)

Point	von Neumann	Harvard
Memory	one memory for code and data	separate code and data memories
Buses	one shared set	two sets: fetch + data **in parallel**
Speed / cost	slower (bottleneck), cheaper	faster, more wiring, costlier
Used in	PCs (main memory), EDVAC, IAS	DSPs, PIC/AVR MCUs, Mark I (1944)

Exam trap: The **von Neumann bottleneck** is the limited transfer rate between CPU and memory over the **single** path (term popularised by John Backus, 1977), not a slow ALU.

Modern CPUs: = **modified Harvard**: split L1 instruction and data caches, but one unified main memory (see Memory Hierarchy).

2 Inside the CPU: ALU, CU, Registers

- **ALU**: arithmetic (add, subtract, increment, compare) and logic (AND, OR, XOR, NOT, shift); sets the flags.
- **Control unit (CU)**: fetches and **decodes** instructions and issues the timing and control signals that make the ALU, registers, memory and I/O act. It directs; the ALU and datapath execute.
- **Registers**: the fastest storage, inside the CPU (next sheet). ALU + registers + internal buses = the **datapath**.

System bus	Direction	Width decides
Address	one-way, CPU →	2^n locations
Data	two-way	word size
Control	mixed lines	RD, WR, INT, clock

Bus maths ✍: 16-bit address bus (8085) → 2^{16} = **64 KB** · 20-bit (8086) → **1 MB** · 32-bit → **4 GB**. MAR width = address bus; MDR width = data bus.

3 The CPU Registers

Reg.	Full name	What it holds / does
PC	Program Counter (x86: IP)	address of the **next instruction**; +1 per fetch
IR	Instruction Register	the instruction **being decoded / executed**
MAR	Memory Address Register	address being read/written; drives address bus
MDR	Memory Data Reg. (MBR)	word just read, or about to be written
AC	Accumulator	implied operand + ALU result (1-address)
SP	Stack Pointer	**top of stack**: PUSH/POP, CALL/RET, interrupts
GPR	R0 … Rn	operands, temporaries (MIPS has 32)
PSW	Flags / status register	condition bits of the last result + control bits

Flag	= 1 when
Z	result is zero
S/N	MSB = 1 (negative)
C	carry out / borrow at MSB
V/O	signed overflow
P	**even** parity (x86)
AC	carry bit 3 → 4 (BCD)

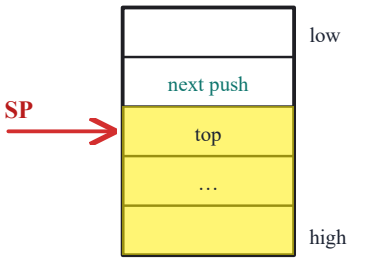
8086 flag register: 16 bits, **9 flags**: 6 status (CF, PF, AF, ZF, SF, OF) + 3 control (TF trap, IF interrupt, DF direction).

Worked ✍️: 8-bit 7FH + 01H = 80H
Z = 0, S = 1, C = 0, **V = 1** (+127 + 1 ≠ −128), AC = 1, P = 0 (one 1-bit)

Stack (LIFO) grows towards lower addresses in most CPUs (x86):

PUSH: SP ← SP − 1, M[SP] ← DR
POP: DR ← M[SP], SP ← SP + 1

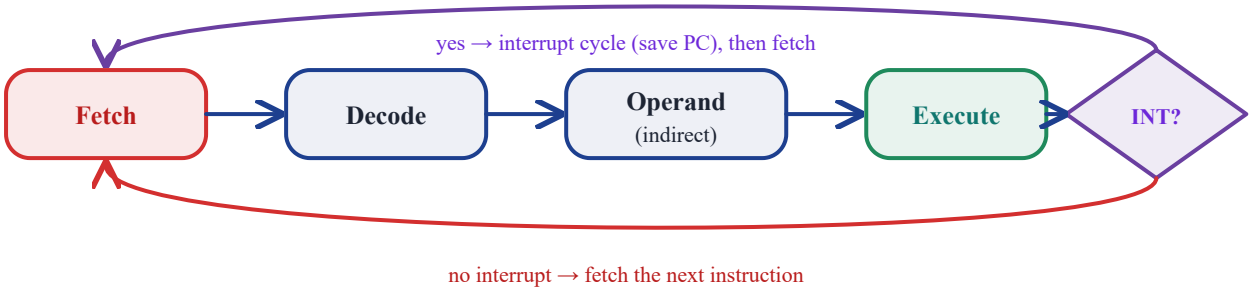
Memory trick: **"PC points, IR holds, MAR addresses, MDR carries."** Fetch always flows PC → MAR → MDR → IR.



stack grows up the page = to lower addresses

4 Instruction Cycle & Micro-operations (RTL)

Instruction cycle: **Fetch → Decode → (fetch operand / indirect) → Execute → check interrupt**, repeated until HALT. Each phase is a sequence of **micro-operations**: elementary register transfers done in one clock pulse.



Phase	Step	Micro-operation (RTL)	Meaning
Fetch	t1	**MAR ← PC**	address out
	t2	**MDR ← M[MAR], PC ← PC + 1**	read, PC + 1
	t3	**IR ← MDR**	load IR
ADD X	t1	MAR ← IR(address)	operand addr.
	t2	MDR ← M[MAR]	read operand
	t3	AC ← AC + MDR	ALU adds
STORE X	t1–t2	MAR ← IR(addr), MDR ← AC; M[MAR] ← MDR	write AC
JMP X	t1	PC ← IR(address)	no memory
Interrupt	t1–t3	MDR ← PC; MAR ← save, PC ← ISR; M[MAR] ← MDR	save PC

RTL notation: ← transfer · comma = same clock pulse · *P: R2 ← R1* = only when control function P = 1 · *M[AR]* = memory word at address AR.

Four kinds of micro-operation: register transfer (R1 ← R2), arithmetic (R3 ← R1 + R2′ + 1), logic (R1 ← R1 ⊕ R2), shift (R1 ← shl R1).

Instruction type	Examples
Data transfer	MOV, LOAD, STORE
Data processing	ADD, SUB, AND, SHL
Program control	JMP, JZ, CALL, RET
I/O	IN, OUT

Exam trap: PC is incremented **during fetch**, so during execute it already holds the address of the **next** instruction. PC-relative addresses are counted from there.

Memory trick: Fetch = "**3 Ts**": address out, data in (+ PC up), into IR. Every instruction pays these 3 pulses.

5 Instruction Formats: 3, 2, 1 and 0 Addresses

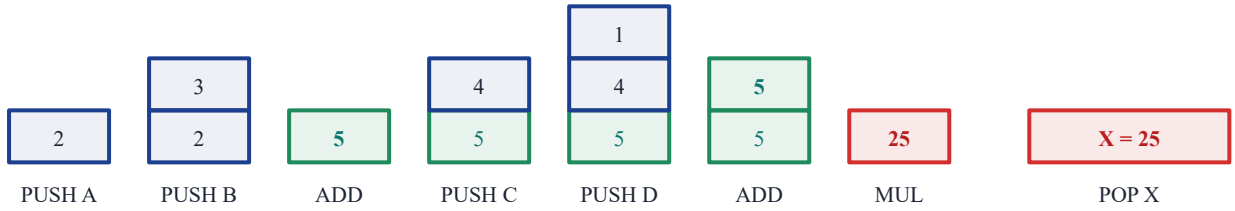
Format: An instruction = **opcode** (what to do) + **mode** bits (how to find the operand) + **address / register fields**. n opcode bits → up to 2^n operations.

Evaluate X = (A + B) × (C + D) on four machine types

3-address	2-address	1-address (accumulator)	0-address (stack)
ADD R1, A, B	MOV R1, A	LOAD A ; AC ← A	PUSH A
ADD R2, C, D	ADD R1, B	ADD B	PUSH B
MUL X, R1, R2	MOV R2, C	STORE T ; temp	ADD
	ADD R2, D	LOAD C	PUSH C
	MUL R1, R2	ADD D	PUSH D
	MOV X, R1	MUL T	ADD
		STORE X	MUL · POP X
3 instr.	**6 instr.**	**7 instr.**	**8 instr.**

0-address code is just the **postfix (RPN)** form AB+CD+× read left to right: operand → PUSH, operator → pop two, push the result.

> fewer addresses → shorter instructions, longer programs



stack trace with A = 2, B = 3, C = 4, D = 1 → (2 + 3)(4 + 1) = 25

Field sizing ✍️: 32-bit instruction, 64 registers, 64K-word memory, format opcode | reg | address.
reg = $\log_2 64$ = 6 bits · address = $\log_2 65536$ = 16 bits
opcode = 32 − 6 − 16 = **10 bits** → 1024 operations

Expanding opcode ✍️: 32-bit instructions, 12-bit addresses, 250 two-address instructions. How many one-address ones are possible?
2-addr opcode = 32 − 24 = 8 bits → 256 patterns; 6 unused
each frees 12 more bits: 6×2^{12} = **24 576**

Exam trap: "0-address" machines still have **PUSH/POP with one address**; only the ALU instructions (ADD, MUL) carry no address.

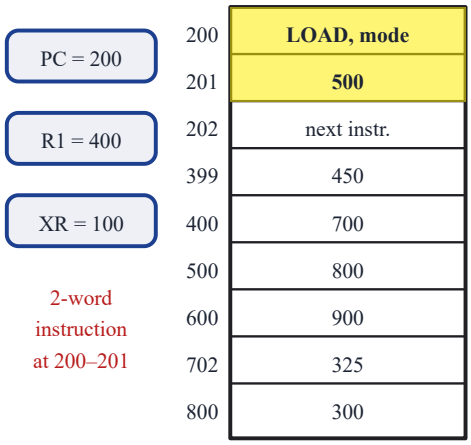
Memory trick: 3-2-1-0 → **3-6-7-8** instructions for this expression: fewer addresses, more instructions.

6 Addressing Modes & Effective Address (EA)

Definition: The addressing mode tells the CPU how to turn the address field into the **effective address (EA)**, the actual memory address of the operand. Modes give pointers, arrays, loops and relocatable code with short instructions.

Mode	Operand / EA	Example	Typical use
Implied	fixed by the opcode	CMA, CLC	accumulator, flags
Immediate	operand **in** the instruction	MOV R1, #25	constants, fastest
Register	operand in register R	ADD R1, R2	no memory access
Register indirect	EA = (R)	MOV R1, (R2)	pointers
Direct (absolute)	EA = address field A	LOAD 500	global variables
Indirect	EA = M[A]	LOAD @500	pointer in memory
Relative	EA = PC + A	BEQ +12	branches, relocation
Indexed	EA = A + (XR)	LOAD 500(X)	arrays
Base register	EA = (BR) + A	[BX + 8]	relocation, segments
Auto-inc / dec	EA = (R), R ± 1 after / before	(R1)+, −(R1)	arrays, stacks

Worked example (classic exam setting)



Mode	EA	AC gets
Direct	500	800
Immediate	201	**500**
Indirect	800	300
Relative (PC = 202)	**702**	325
Indexed (XR = 100)	600	900
Register (R1)	–	400
Register indirect	400	700
Autoincrement	400	700 (R1 → 401)
Autodecrement	399	450 (R1 → 399)

Memory trick: Count the "**hops**" to the operand: immediate 0, direct 1, indirect 2. Register modes trade a memory hop for a register.

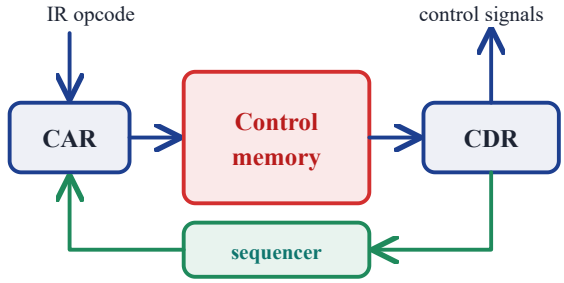
Exam trap: Relative mode adds the **updated** PC (202 here), not 200. Immediate EA is the address of the operand field itself (201).

7 RISC vs CISC

Point	RISC (reduced)	CISC (complex)
Instruction set	few, simple	many, some very complex
Length	**fixed** (32-bit)	**variable** (x86: 1–15 bytes)
Addressing modes	few	many
Memory access	**LOAD/STORE only**	ALU ops may use memory operands
Cycles / instr.	≈ 1, easy to pipeline	many, varies; harder to pipeline
Control unit	hardwired	microprogrammed
Registers	large register file	fewer GPRs
Code / compiler	longer code, smart compiler	compact code, work in hardware
Examples	ARM, MIPS, RISC-V, SPARC	Intel x86, VAX, Motorola 68000

Modern x86 chips decode CISC instructions into RISC-like **micro-ops** internally, so the line has blurred. RISC grew from IBM 801 (1975), Berkeley RISC and Stanford MIPS (early 1980s).

8 Control Unit: Hardwired vs Microprogrammed



microprogrammed CU (Wilkes, 1951)

Hardwired: control signals come from a fixed logic circuit: **sequence counter** + **decoders** + **gates** (or a PLA/FSM).

Microprogrammed: each machine instruction runs a small **microprogram** of microinstructions stored in **control memory** (ROM); CAR = control address register, CDR = control data register.

Point	Hardwired	Microprogrammed
Speed	**faster**	slower (control-memory read each step)
Change / debug	redesign the circuit	rewrite the microcode: **flexible**
Complex ISA	hard, costly	easy
Used by	RISC	CISC

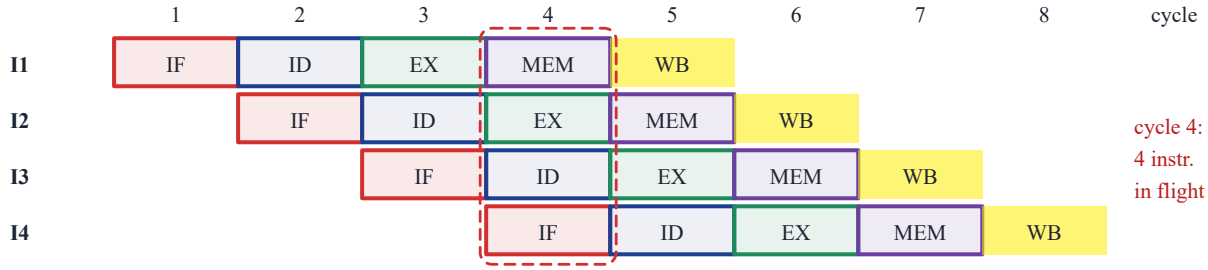
Microinstruction styles: **Horizontal**: 1 bit per control signal, wide, much parallelism, no decoding. **Vertical**: encoded fields, narrow, needs decoders, slower.

Datapath designs: **Single-cycle**: CPI = 1, but clock = slowest instruction (load). **Multicycle**: short clock, CPI 3–5, units reused across steps.

Memory trick: "**Hard = Hurry, Micro = Modify**": hardwired is fast and rigid; microprogrammed is slow and editable.

9 Instruction Pipelining

Definition: Split instruction execution into k stages and **overlap** them, like an assembly line: while one instruction executes, the next is decoded and the one after is fetched. The classic RISC pipeline has **5 stages**: **IF** fetch, **ID** decode + read registers, **EX** ALU, **MEM** memory access, **WB** write back.



space-time diagram: 4 instructions finish in 5 + 3 = 8 cycles instead of 20

Formulas (k stages, n instructions, clock τ):

Pipelined time = **(k + n − 1) τ**

Non-pipelined = n · t_n (t_n = k τ if balanced)

Speedup S = n k / (k + n − 1) → **k**

Efficiency η = n / (k + n − 1) = S / k

Throughput = n / ((k + n − 1) τ) → 1/τ

τ = slowest stage + latch delay

Example 1 ✍️: k = 5, n = 100, one cycle per stage, no stalls → 5 + 99 = **104 cycles**; non-pipelined 500 cycles; S = 500 / 104 ≈ **4.81**; η = 100 / 104 ≈ 96%.

Example 2: unequal stages ✍️: Stage delays 60, 50, 90, 80 ns; latch 10 ns; n = 1000.
τ = 90 + 10 = 100 ns · pipelined = (4 + 999) × 100 = 100 300 ns · non-pipelined = 1000 × (60 + 50 + 90 + 80) = 280 000 ns · S ≈ **2.79**, far below 4 because the stages are unbalanced.

Example 3 ✍️: IF 200, ID 100, EX 200, MEM 200, WB 100 ps. Single-cycle clock = 800 ps (a load uses all five); pipelined clock = 200 ps → ideal speedup 800 / 200 = **4**, not 5.

Exam trap: Pipelining raises **throughput**; it does **not** shorten one instruction's latency (latches even lengthen it a little).

Memory trick: "**Fill once, then one per tick**": k cycles for the first result, then 1 more per instruction → k + n − 1.

Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) between stages hold each instruction's data and control bits; control signals are generated in ID and travel down the pipe with the instruction.

10 Pipeline Hazards & Their Cures

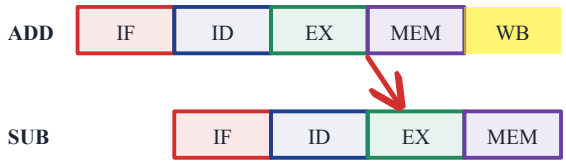
Hazard: A condition that stops the next instruction entering its stage on time, forcing a **stall (bubble)**. Real CPI = 1 + stall cycles per instruction.

Hazard	Cause	Example	Cures
Structural	two instr. need one unit	one memory port	split I/D cache, stall
Data	needs an unfinished result	ADD **R1**,R2,R3 → SUB R4,**R1**,R5	**forwarding**, stall
Control	next PC unknown	BEQ, JMP, CALL	**branch prediction**

Data hazard types (i before j):

- **RAW** read after write: true dependence, the common one
- **WAR** write after read: anti-dependence
- **WAW** write after write: output dependence

WAR and WAW occur only with out-of-order execution; **register renaming** removes them.

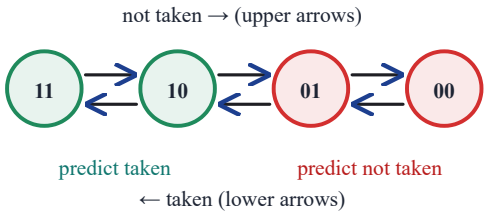


EX/MEM result → ALU input: 0 stalls

forwarding (bypassing)

Exam trap: load-use: LW R1, 0(R2) followed by ADD R3, **R1**, R4 still needs **1 stall** even with forwarding: the loaded value exists only after MEM. Without forwarding a RAW on the next instruction costs 2 stalls (register file written in the first half of WB, read in the second half).

Control hazard cures: static prediction (always not-taken; backward-taken/forward-not-taken), **dynamic** prediction with a 1-bit or **2-bit saturating counter** per branch + a branch target buffer (BTB), **delayed branch** (compiler fills the slot after a branch with a useful instruction), resolving the branch early in ID, flushing wrongly fetched instructions.



2-bit predictor: two wrong guesses to flip

Worked: CPI with stalls ✍️: 5-stage pipeline, 20% branches with a 2-cycle penalty, 30% loads of which 25% are followed by a use (1 stall).
CPI = 1 + 0.2 × 2 + 0.3 × 0.25 × 1 = **1.475** → speedup over unpipelined ≈ 5 / 1.475 ≈ **3.39**.
With a predictor 90% right and a 3-cycle mispredict penalty: CPI = 1 + 0.2 × 0.1 × 3 = **1.06**.

Memory trick: Hazards = "**SDC**": **S**hare a unit, **D**epend on data, **C**hange the PC. Cures in the same order: duplicate, forward, predict.

11 Measuring Performance

The CPU performance equation:

CPU time = IC × CPI × T = IC × CPI / f (IC = instruction count, T = 1/f)

Average CPI = Σ (fraction$_i$ × CPI$_i$) · **MIPS = f / (CPI × 10^6)** = IC / (time × 10^6) · Performance = 1 / time · Speedup = time$_{old}$ / time$_{new}$

Factor	IC	CPI	Clock rate f
Set by	program, compiler, ISA	ISA, pipeline, cache	technology, organisation

Example 1 ✍️: Mix: 50% ALU (CPI 1), 30% load/store (CPI 2), 20% branch (CPI 3).
CPI = 0.5 + 0.6 + 0.6 = **1.7**
IC = 5 × 10^8, f = 2 GHz → time = 5×10^8 × 1.7 / 2×10^9 = **0.425 s**; MIPS = 2000 / 1.7 ≈ **1176**

Example 2 ✍️: Same program. A: 2 GHz, CPI 2.0. B: 3 GHz, CPI 3.6.
A: IC × 2 / 2×10^9 = 1.0 IC ns
B: IC × 3.6 / 3×10^9 = 1.2 IC ns
A is **1.2× faster** despite the slower clock.

Example 3: MIPS ✍️: 400 MHz, CPI 2 → MIPS = 400 × 10^6 / (2 × 10^6) = **200**. Clock period of 400 MHz = 1 / (4 × 10^8) = 2.5 ns.

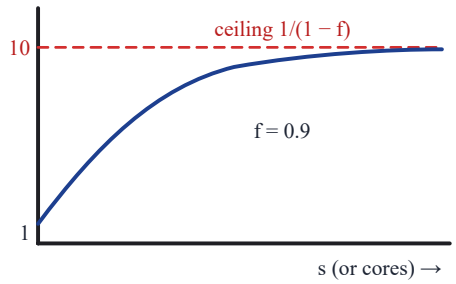
Exam trap: Neither clock rate nor MIPS alone measures performance: MIPS ignores how much work an instruction does (compare RISC and CISC). Only **execution time** of real programs (e.g. SPEC benchmarks) is fair.

12 Amdahl's Law

If a fraction **f** of the time is sped up **s** times:

S = 1 / ((1 − f) + f / s) → max **1 / (1 − f)**

- f = 0.4, s = 10 → 1 / (0.6 + 0.04) = **1.56**
- 90% parallel on 10 cores → 1 / (0.1 + 0.09) = **5.26** (limit 10)
- Overall 2× with f = 0.8: 0.2 + 0.8/s = 0.5 → s = **2.67**



the serial part caps the gain

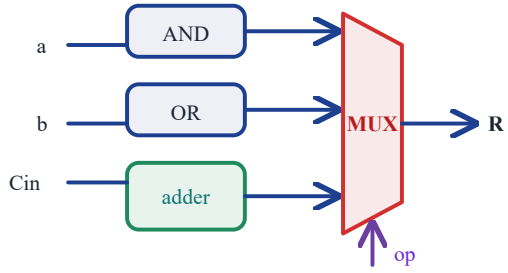
Memory trick: "**Speed up the common case**": the part you do not improve (1 − f) sets the ceiling.

Exam trap: f is a fraction of **time**, not of instructions. With f = 0.5, even s = ∞ gives only 2×.

13 ALU Design & Fixed-Point Arithmetic

A **1-bit ALU slice** computes AND, OR and a full-adder sum in parallel; a **multiplexer** picks one by the operation-select bits. 32 slices with chained carries = a 32-bit **ripple-carry** ALU; a **carry-lookahead** adder removes the slow carry chain.

Subtract: A − B = A + B′ + 1 (invert B, carry-in = 1).



1-bit ALU slice

n-bit two's complement: range $-2^{n-1} \dots 2^{n-1}-1$ (8-bit: **−128 … +127**). Overflow V = C_{in}(MSB) ⊕ C_{out}(MSB): two same-sign operands give an opposite-sign result.

Overflow ✍: 0110 0100 (+100) + 0100 0000 (+64) = 1010 0100 = **−92** in 8 bits. Carry into MSB = 1, out of MSB = 0 → **V = 1**.

Multiplication: shift-and-add; **Booth's algorithm** handles signed (two's complement) operands and skips runs of 1s (pairs 10 → subtract, 01 → add). **Division**: restoring / non-restoring.

14 Floating Point (IEEE 754)



value = (−1)ˢ × 1.F × 2^(E − 127)

Format	Sign	Exponent	Fraction	Bias	Precision
Single (32-bit)	1	**8**	23	**127**	≈ 7 digits, max ≈ 3.4×10^{38}
Double (64-bit)	1	11	52	**1023**	≈ 15–16 digits

Worked: −6.75 in single precision ✍: $6.75 = 110.11_2 = 1.1011 \times 2^2$ · sign = 1 · exponent = 2 + 127 = 129 = 1000 0001 · fraction = 1011 000…0
1 | 10000001 | 10110000000000000000000 = **C0D80000H**

FP add/subtract: ① align: shift the mantissa of the smaller exponent right ② add/subtract mantissas ③ normalise ④ round (guard, round, sticky bits).

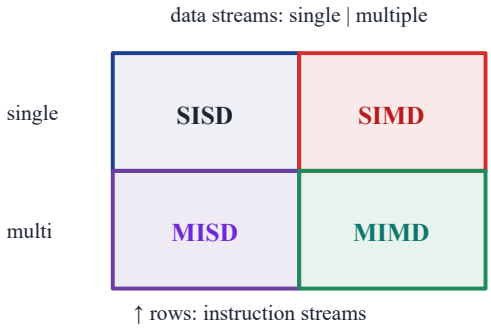
Exam trap: Exponent all 0s = zero or denormal; all 1s = ∞ (F = 0) or NaN (F ≠ 0). The stored exponent is biased: 2^2 is stored as 129, not 2.

Memory trick: "**1-8-23, bias 127**" and "**1-11-52, bias 1023**": bias = 2^(exponent bits − 1) − 1.

15 Beyond the Basic Pipeline

- **Superscalar**: several pipelines; issues **2+ instructions per cycle**, so CPI < 1 (IPC > 1). **Superpipelining**: more, shorter stages. **VLIW**: the compiler packs parallel operations into one long word.
- **Dynamic pipelining** = out-of-order execution: reservation stations (Tomasulo), register renaming, a reorder buffer for in-order commit.
- **Exceptions** (overflow, undefined opcode, page fault) arise inside the CPU; **interrupts** come from I/O. A *precise* exception saves the faulting PC (EPC), records the cause and flushes later instructions.

16 Flynn's Taxonomy (1966)

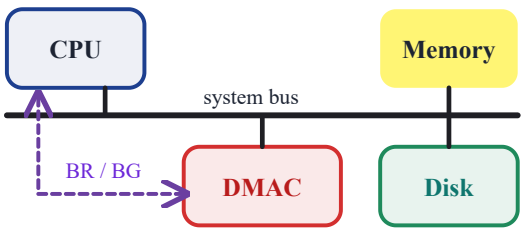


Class	Examples
SISD	classic uniprocessor (von Neumann)
SIMD	**vector & array processors, GPUs**, SSE/AVX
MISD	rare: redundant fault-tolerant systems
MIMD	multicore CPUs, SMPs, clusters

17 I/O Organization

Method	How	CPU involvement
Programmed I/O	CPU polls a status flag (busy-wait)	every word, wastes cycles
Interrupt-driven	device raises INT; CPU runs the ISR	every word, no polling
DMA	DMA controller moves a whole block	**only at start and end**
IOP / channel	I/O processor runs channel programs	least

1. CPU writes start address, word count, direction to the DMA controller
2. DMAC raises **BR** (8086: HOLD); CPU finishes the bus cycle and grants **BG** (HLDA)
3. DMAC moves the data memory ↔ device directly
4. Count = 0 → DMAC releases the bus and interrupts the CPU



Exam trap: DMA **cycle stealing** takes the bus for one word at a time; **burst mode** holds it for the whole block. Either way the data never pass through CPU registers. Channels: **selector** (one fast device at a time) vs **multiplexer** (many slow devices interleaved).

Night-Before-the-Exam Sheet

Fact sheet:

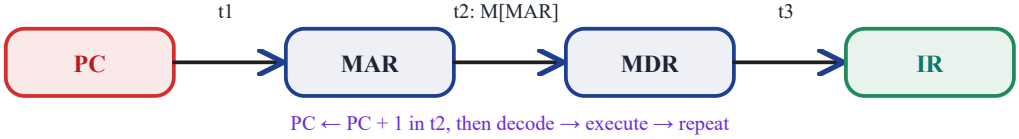
Fetch: MAR←PC; MDR←M[MAR], PC+1; IR←MDR	Address bus n bits → 2^n locations
PC next · IR current · MAR addr · MDR data	8086: 9 flags (6 status + 3 control)
X = (A+B)(C+D): 3/6/7/8 instructions	n opcode bits → 2^n operations
Direct EA = A · Indirect EA = M[A]	Relative = new PC + A · Indexed = A + XR
Pipeline: (k + n − 1) cycles	S = nk/(k + n − 1) → k · η = S/k
τ = slowest stage + latch	CPI_{real} = 1 + stalls per instruction
Time = IC × CPI / f	MIPS = f / (CPI × 10^6)
Amdahl S = 1/((1 − f) + f/s)	Limit 1/(1 − f)
IEEE single 1/8/23, bias 127	Double 1/11/52, bias 1023

Top 14 one-liners for MCQs

1. Stored program: von Neumann, 1945
2. Separate I and D memories = Harvard
3. Next instruction address → PC
4. Operand in the instruction → immediate
5. Array access → indexed mode
6. Stack machine ADD = 0-address
7. RISC: fixed length, load/store
8. Control memory → microprogrammed
9. Hardwired CU: faster, less flexible
10. RAW = data hazard → forwarding
11. Branch → control hazard → prediction
12. Load-use: 1 stall even with forwarding
13. GPU / vector = SIMD; multicore = MIMD
14. DMA: CPU only at start and end

Top traps: CU decodes and directs, it does not "execute" · cache is a memory level, not a CPU unit · relative mode uses the updated PC · immediate EA = address of the operand field · pipelining improves throughput, not latency · speedup < k when stages are unequal · MIPS and GHz mislead · 7FH + 01H sets V, not C

All tricks together: PC points, IR holds, MAR addresses, MDR carries · fetch = 3 Ts · 3-2-1-0 → 3-6-7-8 · count the hops · Hard = Hurry, Micro = Modify · fill once, then one per tick · hazards SDC → duplicate, forward, predict · speed up the common case · 1-8-23 bias 127



PC ← PC + 1 in t2, then decode → execute → repeat

the fetch path you must be able to write in RTL

✓ Practice MCQs (exam pattern, 10 marks)

1. A von Neumann machine keeps instructions and data in:
(a) two memories (b) one memory
(c) ROM only (d) registers

2. Which register holds the instruction being decoded?
(a) PC (b) MAR
(c) IR (d) SP

3. MOV R1, #25 uses which addressing mode?
(a) Direct (b) Indirect
(c) Indexed (d) Immediate

4. 5-stage pipeline, 100 instructions, no stalls. Cycles?
(a) 104 (b) 500
(c) 105 (d) 100

5. Address fields in a stack machine's ADD:
(a) 1 (b) 0
(c) 2 (d) 3

6. A 2 GHz CPU with average CPI 4 gives:
(a) 800 MIPS (b) 8000 MIPS
(c) 500 MIPS (d) 200 MIPS

7. Half the run time is made 2× faster. Overall speedup:
(a) 2 (b) 1.5
(c) 1.25 (d) 1.33

8. Operand forwarding mainly reduces:
(a) data hazards (b) structural
(c) control hazards (d) cache misses

9. A GPU / vector processor is in Flynn's class:
(a) SISD (b) SIMD
(c) MISD (d) MIMD

10. In DMA the CPU is involved:
(a) every word (b) never
(c) at start and end (d) on errors only

Answer key → 1-b · 2-c · 3-d · 4-a · 5-b · 6-c · 7-d · 8-a · 9-b · 10-c

Written question: A 5-stage pipeline has stage delays 150, 120, 160, 140, 130 ps and a 20 ps latch delay. For 1000 instructions find (a) the clock period, (b) pipelined and non-pipelined times, (c) the speedup, (d) the time if 20% are branches with a 2-cycle penalty. 8 marks

1. τ = slowest stage + latch = 160 + 20 = **180 ps** 1
2. Pipelined: (5 + 999) × 180 = **180 720 ps**; non-pipelined: 1000 × 700 = **700 000 ps** 3
3. Speedup = 700 000 / 180 720 ≈ **3.87** (limit 3.89, not 5: unequal stages) 2
4. Stalls = 1000 × 0.2 × 2 = 400 → (1004 + 400) × 180 = **252 720 ps**, S ≈ 2.77 2

Before the exam ✓

- ✓ Fetch, ADD, interrupt cycles in RTL
- ✓ EA and operand for every mode
- ✓ (A+B)(C+D) in 3/2/1/0-address code
- ✓ Pipeline, CPI, MIPS, Amdahl sums

Also practise (written):
① Hardwired vs microprogrammed
② RISC vs CISC, six points
③ Three hazards with cures
④ 12.375 in IEEE 754